

Industrial Internship Report on Flask Job Application Tracker(JobTrack)

Prepared by
Himanshu sah

Executive Summary

JobTrack is a server-rendered web application developed as a Python development internship final project. It helps a user maintain job applications, update their progress, and identify upcoming deadlines. The project turns a practical organizational problem into a working application with account-based access and persistent storage.

The implementation uses Flask, Jinja, Bootstrap, SQLite through Flask-SQLAlchemy, Flask-Login, and Flask-WTF. Users can register, log in, add or update an application, view notes, and confirm deletion. A dashboard presents total applications and counts for Applied, Interviewing, Offered, and Rejected, alongside company/title search, status filtering, and active deadlines.

The system separates authentication and tracker features into blueprints and uses an application factory for configuration and test isolation. Passwords are hashed; application queries enforce ownership; forms validate input and include CSRF protection. The completed local project includes repeatable fictional demo data, setup instructions, screenshots, and a recorded browser walkthrough.

Verification includes 18 passing pytest cases and browser workflow checks on desktop and mobile layouts. This report describes those functional results without treating them as load-testing evidence. The project is intended for a local internship demonstration; public hosting, notifications, and advanced analytics remain future work.

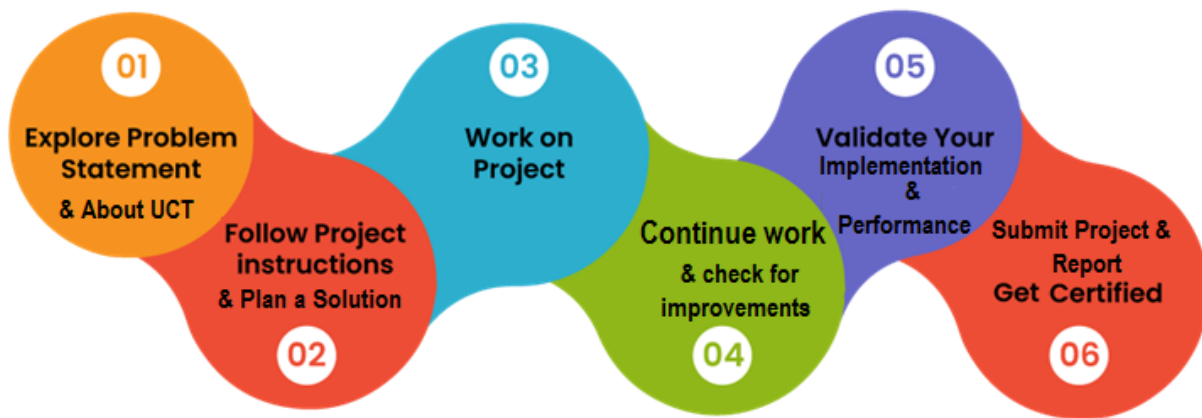
TABLE OF CONTENTS

1	Preface	3
2	Introduction	4
2.1	About UniConverge Technologies Pvt Ltd.....	4
i.	UCT IoT Platform	4
2.2	About upskill Campus (USC)	8
2.3	The IoT Academy	10
2.3	Objective	10
2.4	Reference	10
2.5	Glossary	11
3	Problem Statement.....	12
4	Existing and Proposed solution	13
4.1	Existing Approaches.....	Error! Bookmark not defined.
4.2	Proposed Solution and Value Addition	13
4.3	Code submission (Github link)	13
4.4	Report submission (Github link) :.....	13
5	Proposed Design/ Model	14
5.1	High-Level Diagram.....	14
5.2	Low Level Diagram.....	15
	Stock Movement Algorithm	Error! Bookmark not defined.
5.3	Interfaces (if applicable)	16
5.4	Application Screenshots	Error! Bookmark not defined.
6	Performance Test.....	Error! Bookmark not defined.
6.1	Test Plan/ Test Cases	Error! Bookmark not defined.
6.2	Test Procedure.....	Error! Bookmark not defined.
6.3	Performance Outcome	Error! Bookmark not defined.
7	My learnings	Error! Bookmark not defined.
8	Future work scope	Error! Bookmark not defined.

1 Preface

An internship project connects programming concepts with a practical software workflow. Inventory management is a suitable problem because it requires persistent records, reliable validation, controlled access, and traceable changes rather than only a visual interface.

Stockroom was selected as the final project for the Python development domain. Its scope covers product records, stock movements, low-stock alerts, administrator and staff roles, and reporting. The implementation is designed to be understandable and practical for a one-week final-project preparation window.



Acknowledgement: I thank upskill and the internship team for their guidance and support.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



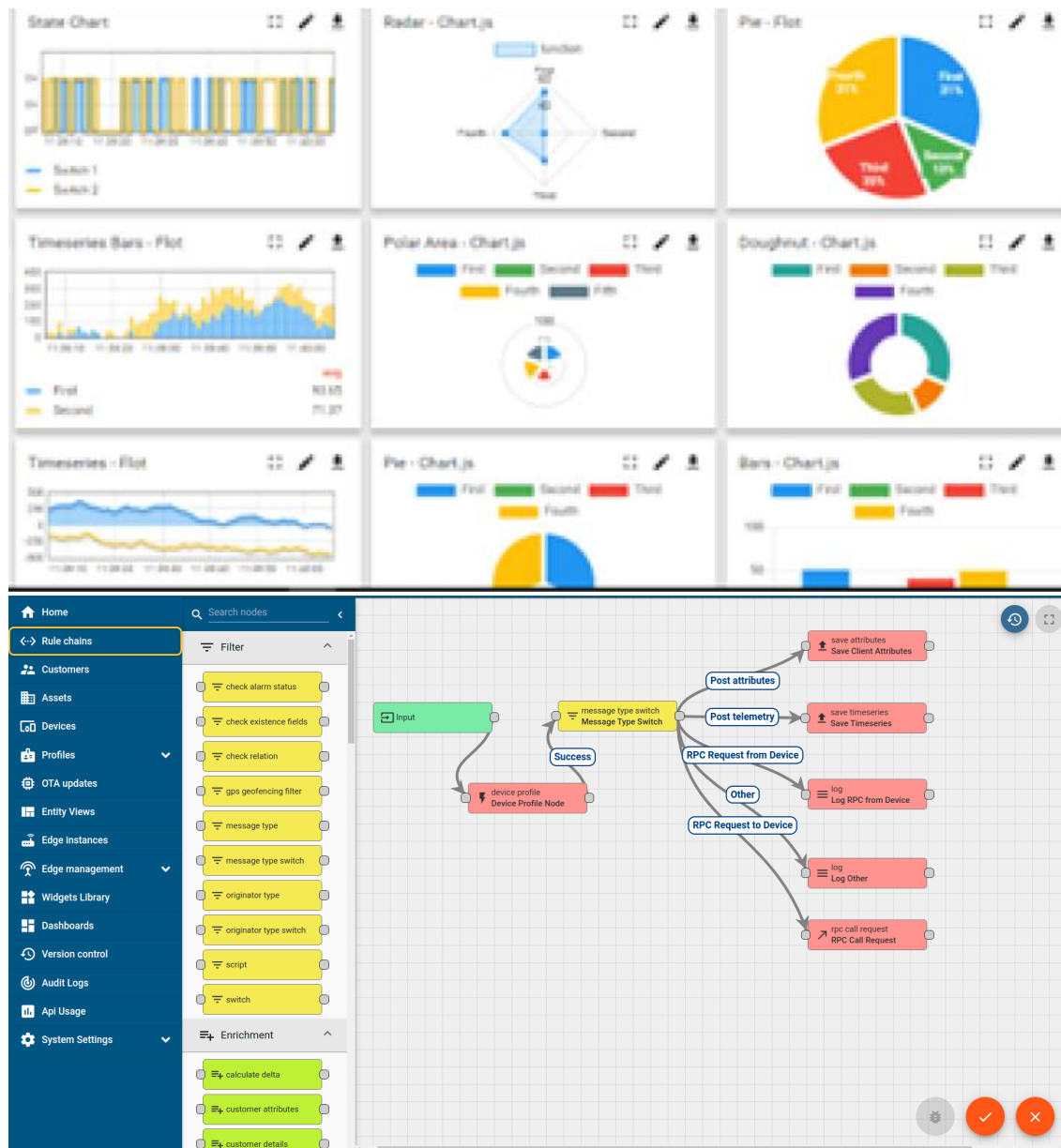
i. UCT IoT Platform (uct Insight)

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



FACTORY WATCH

ii. Smart Factory Platform ()

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



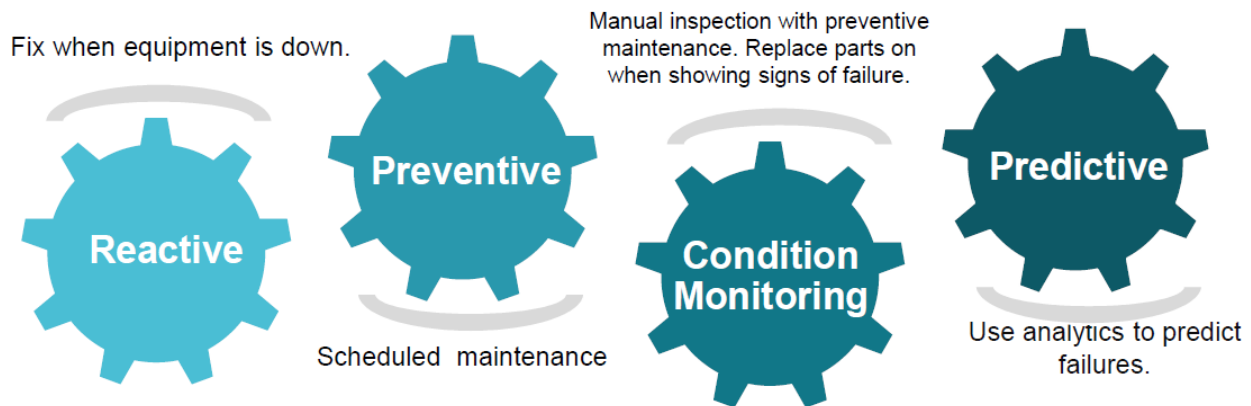


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

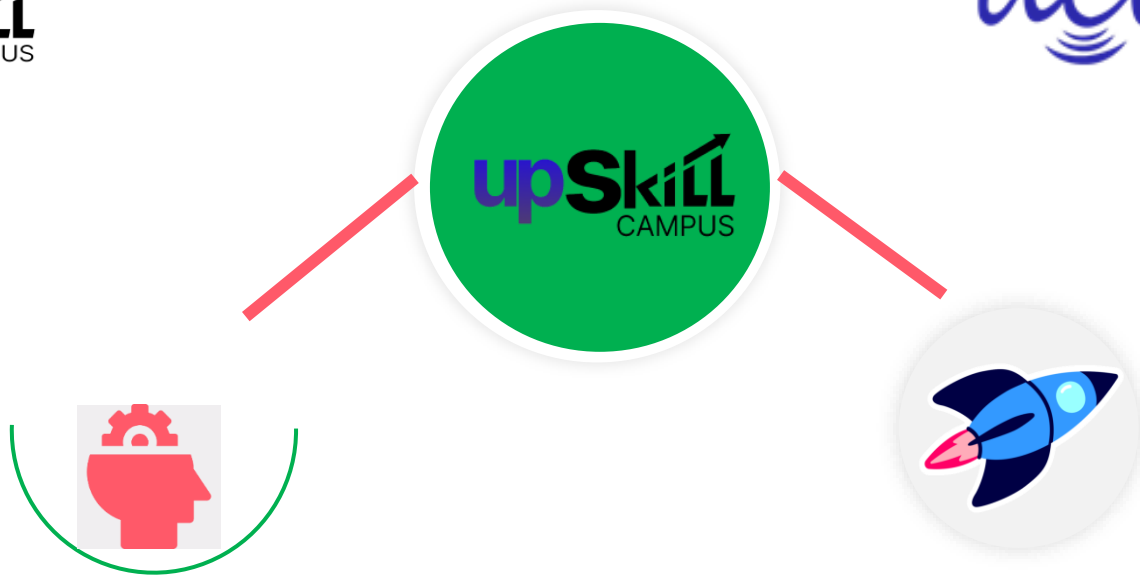
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

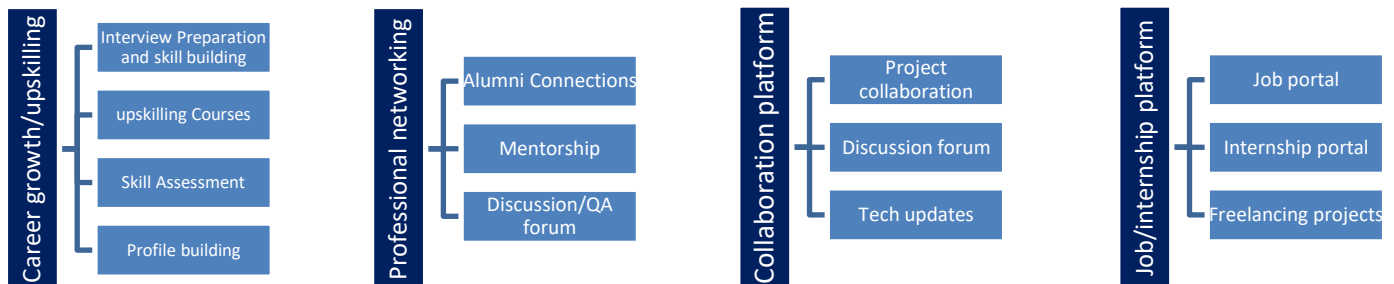
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

<https://www.upskillcampus.com/>



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.3 Objective

Job applications often involve several companies, role titles, posting links, deadlines, and interview notes. A dedicated tracker can bring these details into one place. JobTrack applies Python to this everyday problem through a web interface that requires no separate frontend framework or REST client.

- Develop a complete Python web application that solves a clear organizational problem.
- Learn Flask routes, blueprints, templates, forms, and the application factory pattern.
- Store user and application data in a relational SQLite database.
- Implement authentication and restrict each account to its own records.
- Provide usable application management, search, status counts, and deadline visibility.
- Verify workflows and document installation, demonstration, and limitations.

2.4 Reference

[1] UniConverge Technologies, About Us
<https://www.uniconvergetech.in/about-us>

[2] upskill Campus, Industrial Internship Program
<https://learn.upskillcampus.com/s/pages/industrial-internships>

[3] Flask, Application Setup
<https://flask.palletsprojects.com/en/stable/tutorial/factory/>

[4] Flask, Testing Flask Applications
<https://flask.palletsprojects.com/en/stable/testing/>

[5] Project source and local artifacts
README.md; requirements.txt; jobtrack/; tests/; docs/VALIDATION.md; docs/screenshots/

2.5 Glossary

Term	Meaning in this project
Flask	Python web framework handling requests, routes, and rendered responses.
CRUD	Create, read, update, and delete application records.
ORM	Object-relational mapping between Python models and database tables.
CSRF	Cross-site request forgery; token protection checks state-changing forms.
Blueprint	Flask module grouping related routes.
Jinja	Template engine rendering server-side HTML.
SQLite	File-based relational database used for local storage.
HTTP / HTTPS	Protocols permitted for an optional job-posting URL.

3 Problem Statement

A student or early-career job seeker may apply for several positions and maintain the information in scattered notes, emails, or spreadsheets. Without a consistent workflow, it becomes difficult to find a posting link, check application progress, identify approaching deadlines, or prepare from previous interview notes.

The problem addressed by JobTrack is to provide a single, account-based place to record these opportunities and review progress. The user should be able to maintain accurate data without needing database knowledge and should be prevented from accessing another account's applications.

3.1 Functional requirements

Requirement	Acceptance criterion
Accounts	Register with email/password, log in, and log out.
Application management	Create, view, edit, and delete after confirmation.
Required data	Company, job title, application date, and one supported status.
Optional data	HTTP/HTTPS posting URL, deadline, and notes.
Dashboard	Total and per-status counts for the complete account.
Discovery	Company/title text search and optional status filter.
Deadline visibility	Today through seven days ahead; exclude Offered and Rejected.

3.2 Constraints and acceptance criteria

The implementation is sized for a one-week final-project effort and a local demonstration. It uses server-rendered pages and SQLite, avoiding the setup overhead of a separate frontend and database server. Application dates use the host machine's local calendar date.

A reviewer should be able to follow the README, initialize the database, register or use the fictional demo account, and complete application management without errors. Invalid forms should remain editable with feedback, and unauthorized record access should fail. The interface should fit desktop and mobile screens.

Email reminders, AI functions, scraping, file uploads, and hosting are outside the implemented scope. No financial data, resumes, or real applicant documents are required for the demonstration.

4 Existing and Proposed solution

4.1 Existing approaches

The following comparison describes typical workflow choices rather than a benchmark of commercial products. Notes and spreadsheets are easy to start but require manual organization; email folders retain correspondence but do not provide one structured application record. JobTrack focuses on a small, dedicated workflow suitable for demonstrating Python development.

Approach	Typical strength	Limitation for this workflow
Notes / documents	Flexible and quick to write.	Statuses, dates, and links are not consistently structured.
Spreadsheet	Can store rows and use filters.	Forms, authenticated ownership, and deadline logic require extra setup.
Email folders	Retain recruiter conversations.	Application details and next steps remain spread across messages.
JobTrack	Structured forms, account-specific records, dashboard, and deadlines.	Local-demo scope; no external integrations or notifications.

4.2 Proposed Solution and Value Addition

JobTrack stores each application with an owning user and a consistent set of fields. The dashboard summarizes progress while the detail page preserves posting links and notes. Separate create/edit forms validate input before committing data. A dedicated confirmation page reduces accidental deletion.

The main value is a complete workflow: capture an opportunity, review it, update progress, and prepare for deadlines. Summary counts remain account-wide even when the list is filtered. This prevents a filtered view from changing the user's understanding of their overall application activity.

4.3 Code submission (Github link)

(GitHub link): [[UpSkill/jobtrack at main · Himanshu2233/UpSkill](https://github.com/Himanshu2233/UpSkill)]

4.4 Report submission (Github link) :

(GitHub link): [[UpSkill/docs/report/JobTracker_Himanshu_USC_UCT.pdf at main · Himanshu2233/UpSkill](https://github.com/Himanshu2233/UpSkill/blob/main/docs/report/JobTracker_Himanshu_USC_UCT.pdf)]

5 Proposed Design/ Model

5.1 High-Level Diagram

The browser sends requests to Flask. Authentication and tracker blueprints implement the routes; form classes validate submissions; SQLAlchemy reads and writes the SQLite database. Jinja templates and local Bootstrap/CSS files produce the user interface. Flask's application factory initializes configuration and extensions [3].

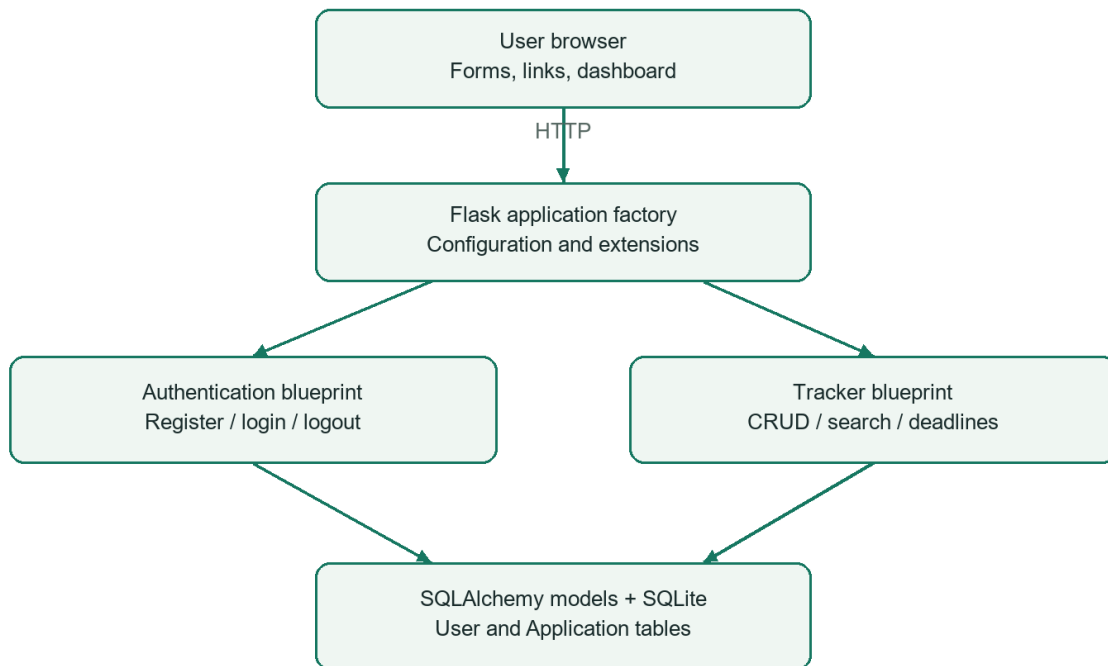


Figure 1. High-level architecture of JobTrack.

Responses are server-rendered HTML; the project exposes no separate public REST API. Static styles are bundled locally for an offline-capable demonstration. The database and generated local secret key reside in the ignored instance directory.

5.2 Low Level Diagram

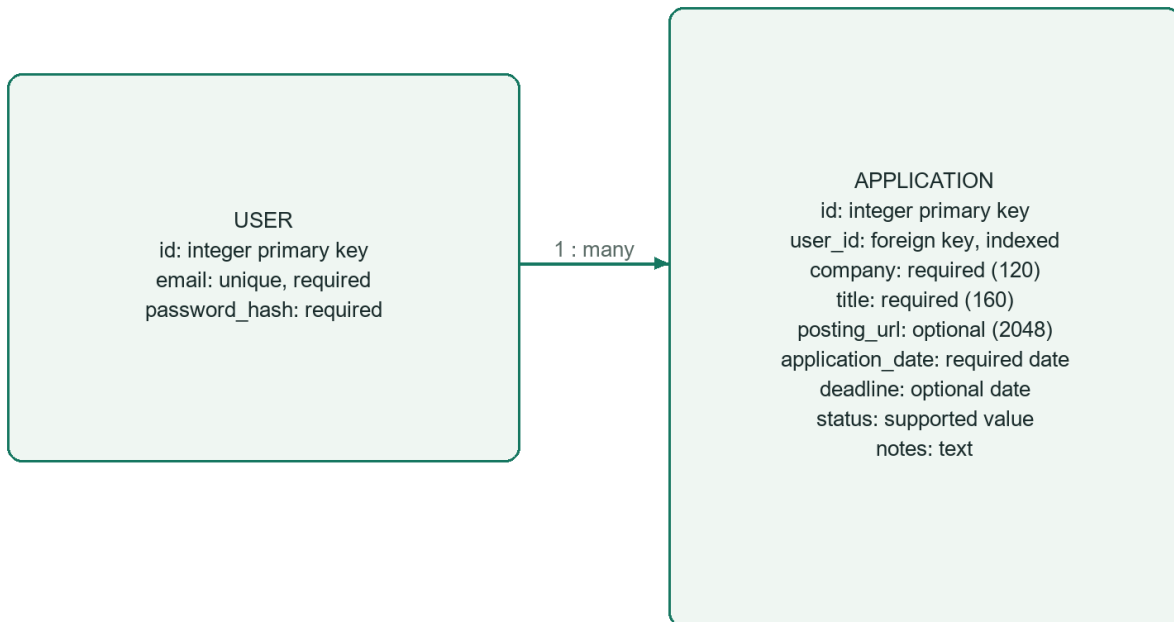


Figure 2. User-to-Application relationship and stored fields.

One user can own many applications. The indexed user_id supports account-specific queries. A database check constraint limits statuses to Applied, Interviewing, Offered, and Rejected. Passwords are stored as PBKDF2-SHA256 hashes using Werkzeug; the original password is not stored.

The dashboard selects the current user's records, calculates total and status counts, and then applies search/status criteria to the displayed list. Applications are ordered by descending application date and ID. Upcoming deadlines are sorted by date and ID and include the two boundary dates, today and today plus seven days.

For view, edit, and delete operations, the record lookup includes both the requested ID and current user ID. A missing or foreign-owned record produces HTTP 404. This same ownership check applies to GET and POST operations.

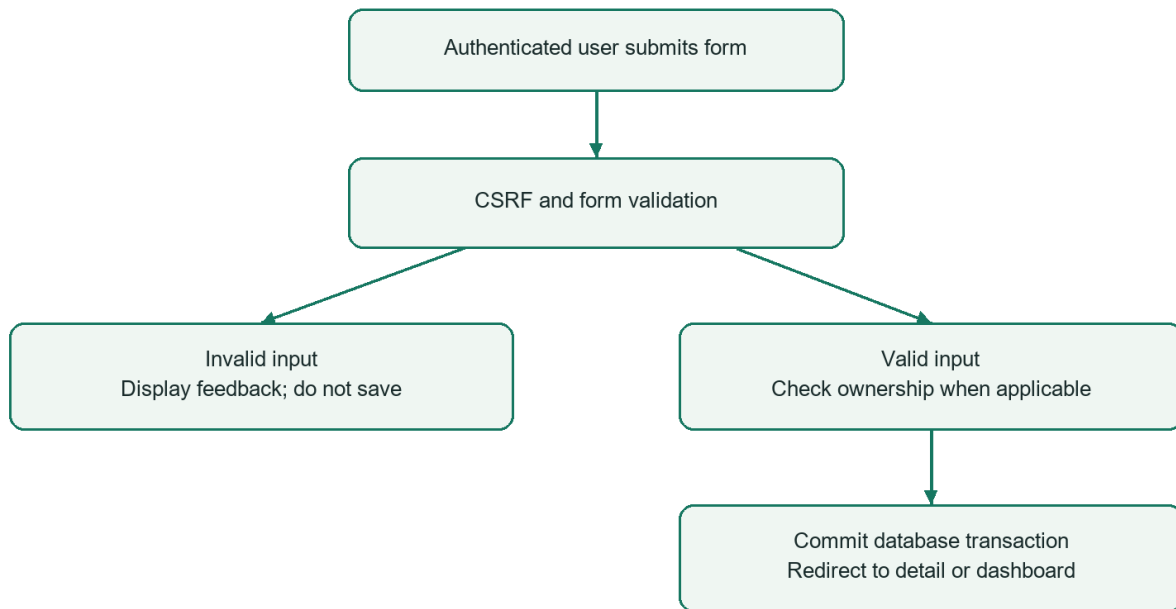


Figure 3. Form submission and persistence flow.

5.3 Interfaces (if applicable)

Method / path	Purpose
GET /	Protected dashboard; accepts q and status query parameters.
GET, POST /auth/register	Account registration form and submission.
GET, POST /auth/login	Login form and submission.
POST /auth/logout	CSRF-protected logout.
GET, POST /applications/new	Create an application.
GET /applications/<id>	Display an owned record.
GET, POST /applications/<id>/edit	Edit an owned record.
GET, POST /applications/<id>/delete	Show confirmation; delete on valid POST.

Company and job title are trimmed and required. Email syntax and password length are validated, and registration requires matching passwords. Application dates must parse as valid calendar dates. Optional posting URLs require an HTTP or HTTPS scheme and hostname. Notes are limited to 10,000 characters by form validation.

CSRF protection is enabled globally. Jinja escapes displayed user content, while job links open with noopener and noreferrer. Sessions use HttpOnly and SameSite=Lax cookies, and responses include no-store, nosniff, and SAMEORIGIN headers. These controls support the local workflow; no public deployment security review is claimed.

The authentication screen provides email/password login and a link to registration. Form errors remain visible next to invalid input.

The screenshot shows the JobTrack login page. At the top left is the JobTrack logo, and at the top right is a 'Log in' link. A light blue banner at the top says 'Please log in to access this page.' The main content area is divided into two sections. On the left, under the heading 'YOUR NEXT CHAPTER STARTS HERE', is a large green headline 'Your job search, in a better place.' followed by the text 'Keep every opportunity, deadline, and next step together. Spend less time organizing and more time moving forward.' and a green button that says 'One dashboard. A clearer direction.' On the right, under the heading 'LET'S GET STARTED', is a 'Welcome back' section with the text 'Your next opportunity is waiting.' Below this are two input fields labeled 'Email address' and 'Password'. A green 'Log in' button is positioned below the password field. At the bottom of this section is a link that says 'New here? Create an account'. The footer contains the text 'Small steps. New opportunities.' on the left and 'Made with Python & Flask' on the right.

Figure 4. Login interface.

The dashboard combines account-wide statistics, searchable application records, and the next seven days of active deadlines.

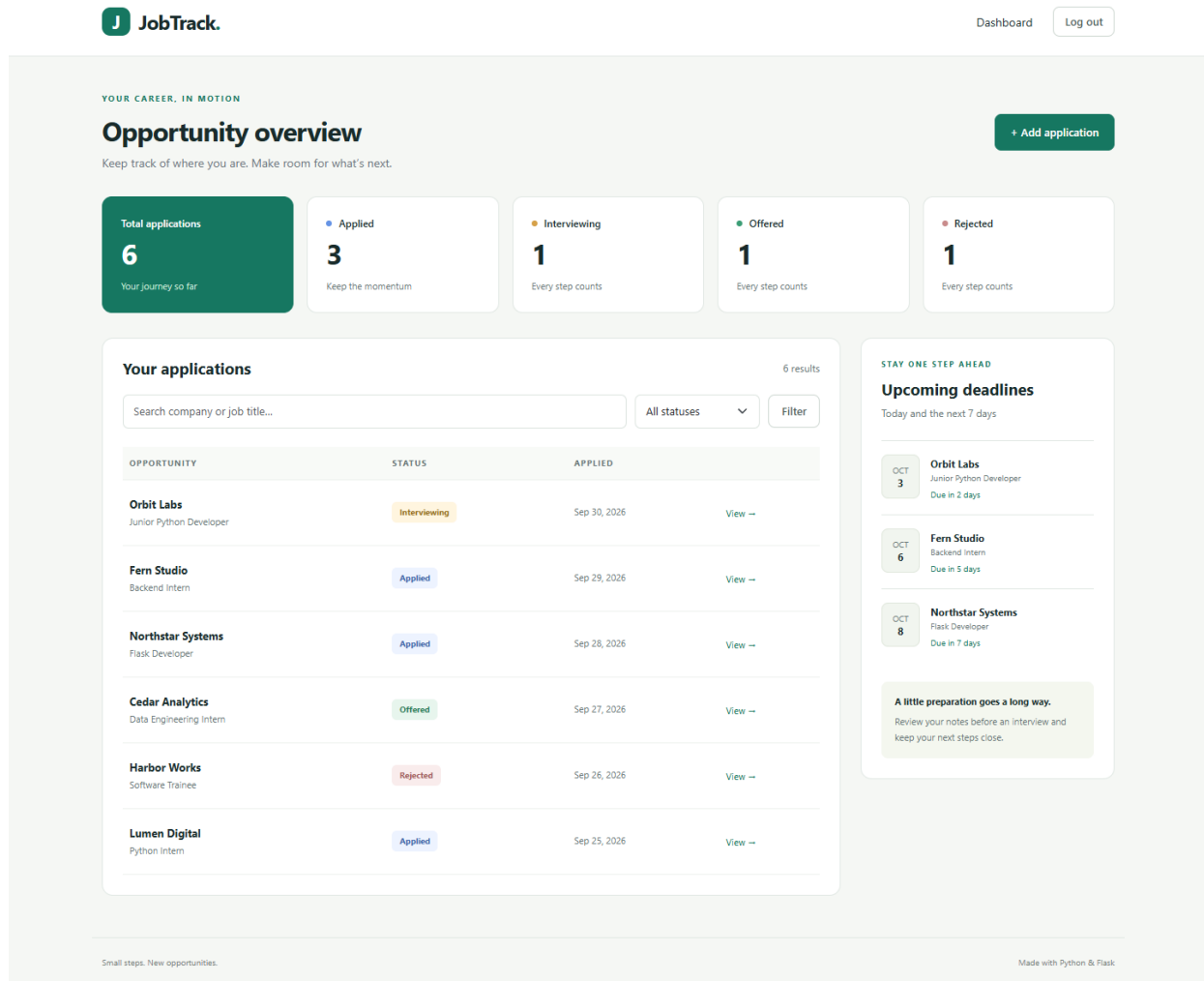


Figure 5. Desktop dashboard with fictional demo data.

The create/edit form captures company, role, posting URL, dates, status, and notes using labeled fields.

The screenshot shows the 'Add an opportunity' form in the JobTrack application. The form is titled 'Add an opportunity' with a subtitle 'The details today. A clearer picture tomorrow.' and a motivational tagline 'MAKE YOUR NEXT MOVE'. The form contains several labeled fields: 'Company' (Willow Demo Studio), 'Job title' (Junior Python Developer), 'Posting URL (optional)' (https://example.com/jobs), 'Application date' (10/01/2026), 'Deadline (optional)' (mm/dd/yyyy), 'Status' (Applied), and 'Notes (optional)' (Prepare Flask portfolio. Ask about mentorship and code reviews.). At the bottom of the form are 'Save application' and 'Cancel' buttons. The top navigation bar includes the JobTrack logo, a 'Dashboard' link, and a 'Log out' button. The footer contains the text 'Small steps. New opportunities.' and 'Made with Python & Flask'.

Figure 6. Application entry form.

The detail page displays dates, posting link, status, and notes, with explicit edit and delete actions.

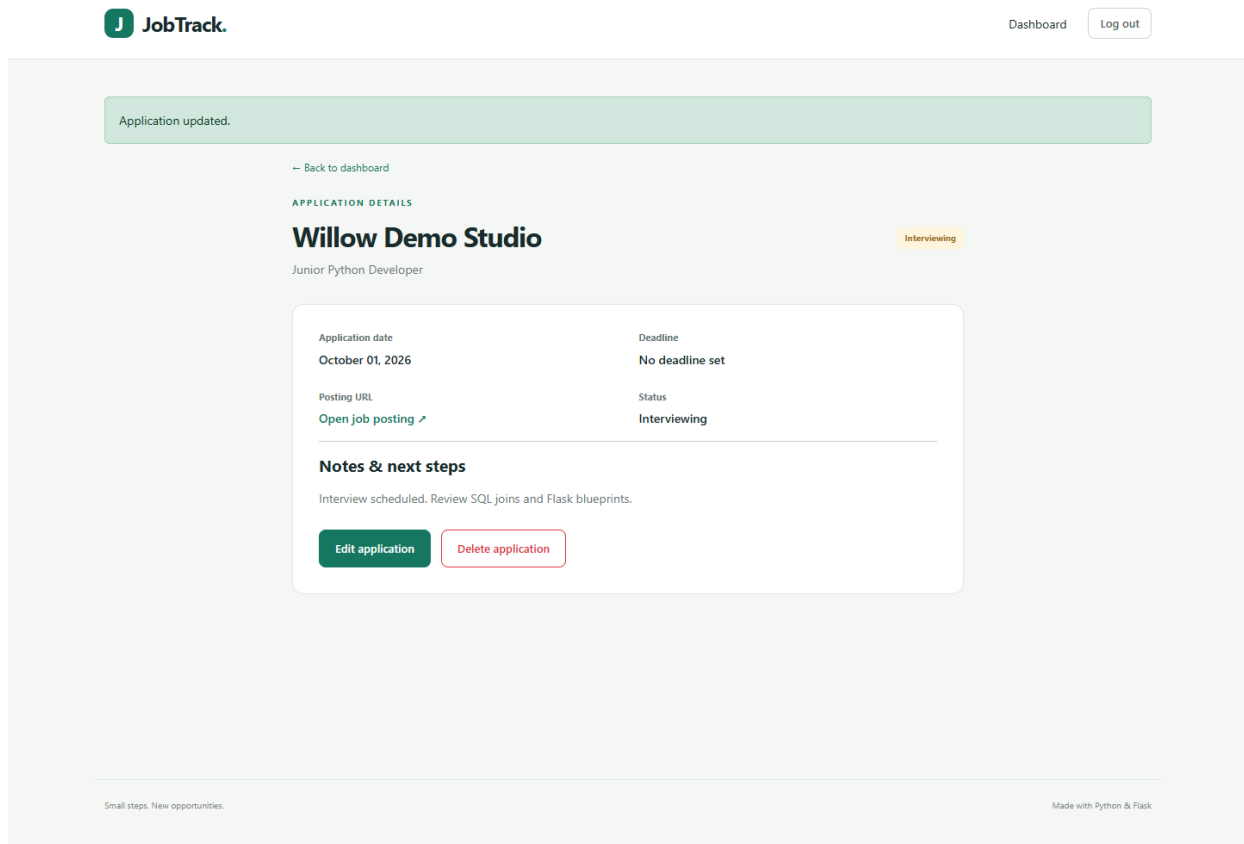


Figure 7. Application details after editing.

The confirmation page supports both cancellation and a CSRF-protected deletion submission.

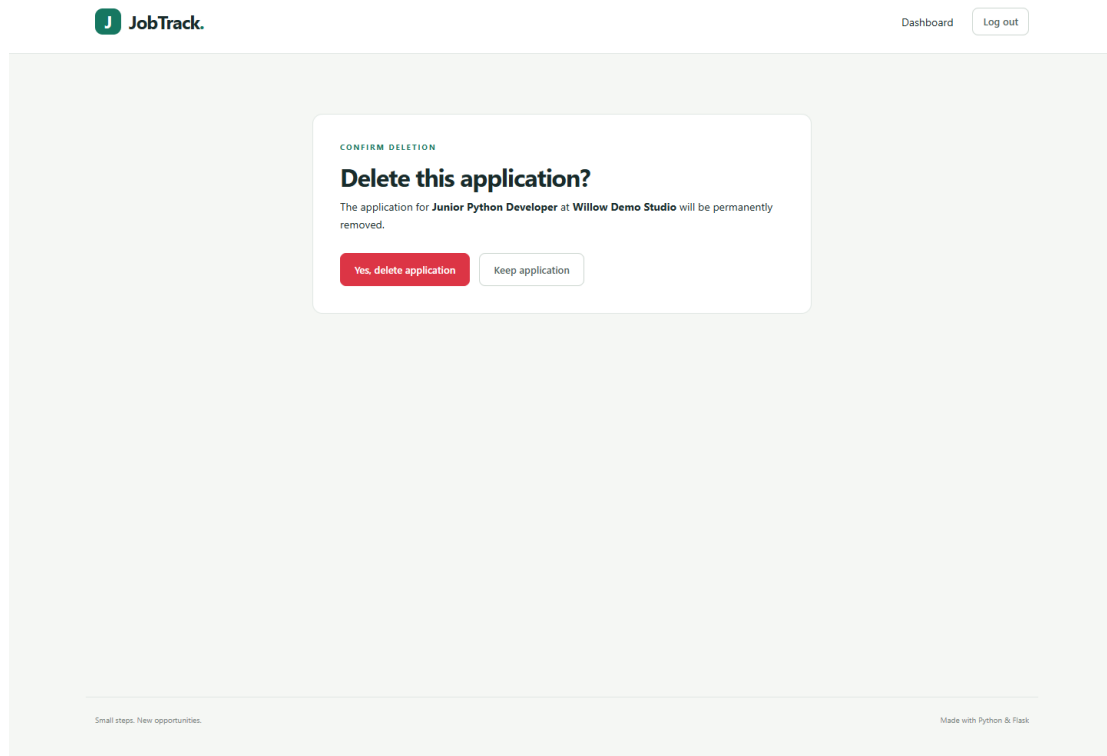


Figure 8. Deletion confirmation.

On smaller screens, statistic cards wrap and the deadline panel follows the application list. Browser checks found no document horizontal overflow.

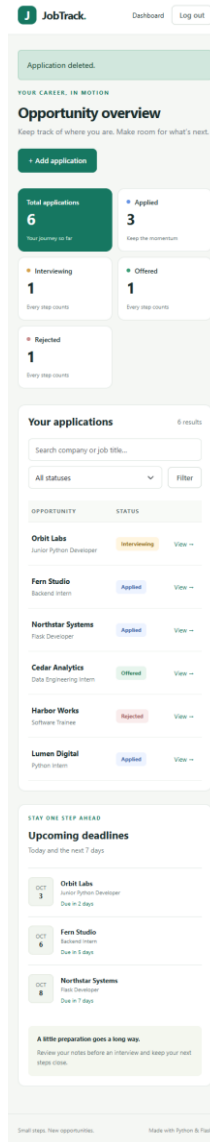


Figure 9. Responsive dashboard at a 390-pixel viewport.

A newly registered user sees an invitation to add an application and cannot see the demo account's records.

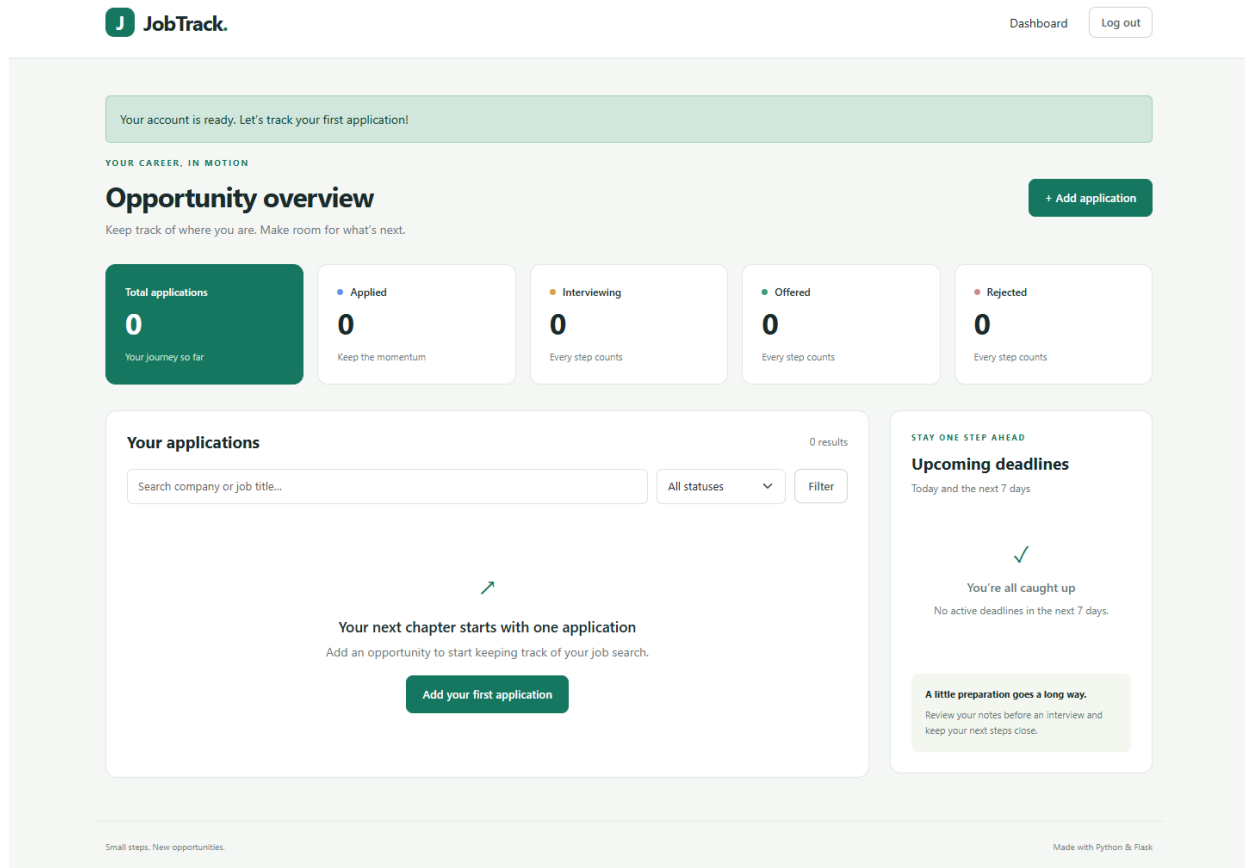


Figure 10. New account with an empty dashboard.

5.4 Implementation and local execution

Component	Responsibility
Application factory / extensions	Configure database, session handling, login, and CSRF integrations.
Models / forms	Define persisted entities and validate submitted values.
Authentication blueprint	Register accounts, verify passwords, and manage sessions.
Tracker blueprint	Enforce ownership and implement CRUD/dashboard behavior.
Templates / static assets	Render the responsive interface with Jinja and local Bootstrap.
CLI / tests	Initialize database, seed fictional records, and verify behavior.

Validated environment: Windows, Python 3.8.10, and Microsoft Edge. Direct dependency versions are Flask 3.0.3, Werkzeug 3.0.6, Flask-SQLAlchemy 3.1.1, SQLAlchemy 2.0.36, Flask-Login 0.6.3, Flask-WTF 1.2.1, WTForms 3.1.2, email-validator 2.2.0, pytest 8.3.5, and python-dotenv 1.0.1. Bootstrap 5.3.3 is bundled locally.

A reviewer can run the project in PowerShell with the following commands. The virtual environment and database are created locally; initialization does not delete existing tables or records.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m flask --app jobtrack init-db
.\.venv\Scripts\python.exe -m flask --app jobtrack seed-demo
.\.venv\Scripts\python.exe -m flask --app jobtrack run
```

Open <http://127.0.0.1:5000>. The fictional demo login is `demo@jobtrack.example` with password `DemoPass123!`. The seed command creates six applications only when the demo account is absent; rerunning preserves existing demo data. These credentials are demonstration data only.

`SECRET_KEY` and `DATABASE_URL` may be configured through `.env`. If no key is provided, the application creates a persistent local key in `instance/secret.key`. The default database is `instance/jobtrack.sqlite`. Test configuration replaces this database with a temporary SQLite file.

6 6 Performance Test

The verification objective was to establish functional correctness, account isolation, and interface usability for a small local demonstration. The available results do not measure production throughput or latency. Functional test execution time is reported separately from application response time.

6.1 6.1 Test Plan / Test Cases

Test group	Scenario and expected result	Observed
Authentication	Protected pages redirect; registration hashes passwords; valid login succeeds; invalid login fails.	Pass
Registration validation	Reject malformed email, short password, mismatched confirmation, and duplicate email.	Pass
CRUD / confirmation	Create, read, edit, cancel deletion, then confirm deletion; deleted URL returns 404.	Pass
Field validation (9 cases)	Reject blank company/title, malformed dates, invalid URL forms, unsupported status, and overlong notes.	Pass
Content safety	Accept optional HTTP/HTTPS link; escape script text in notes.	Pass
Ownership	Second account cannot list, read, edit, or delete the first account's records.	Pass
Search / counts / dates	Verify search/filter, literal wildcard search, full-account counts, and deadline boundaries.	Pass
CSRF	Reject tokenless registration, create, and logout; permit valid token registration.	Pass
Empty/error pages	Show empty dashboard, deadline empty state, form page, and 404.	Pass
CLI seeding	Initialize and rerun seed without duplicating demo users or applications.	Pass

The table groups related checks. The suite collects 18 pytest cases because field validation is parameterized into nine separate cases. This count is not a code-coverage percentage or a claim that every possible input was tested.

6.2 6.2 Test Procedure

Automated tests use the application factory with testing configuration, a temporary SQLite database, and a test client. Tables are created for each fixture and removed afterward. Most functional tests disable CSRF to focus on workflows; the dedicated CSRF test enables it and verifies both rejection and a valid token path. Flask's test client supports this isolated approach [4].

```
.\.venv\Scripts\python.exe -m pytest -q
```

The latest report-generation verification run completed with 18 tests passing in 7.41 seconds on the local environment. Test duration depends on the machine and password hashing overhead; it should not be presented as a web-response benchmark.

Browser checks used headless Microsoft Edge and Playwright with a 1440 x 1000 desktop viewport and a 390 x 844 mobile viewport. The walkthrough performed demo login, combined search/filter, reset, creation, editing, deletion cancellation and confirmation, logout, registration, empty-state display, and direct cross-account URL rejection. It also captured screenshots and a silent video. No JavaScript page errors were observed.

6.3 6.3 Performance Outcome

Measure / constraint	Result or limitation
Automated functional verification	18 passed; no failed cases in the latest run.
Dependency consistency	pip check reported no broken requirements in the validated environment.
Account isolation	Foreign-owned detail/edit/delete requests rejected in tests; browser read rejection also verified.
Responsive layout	No document horizontal overflow on tested dashboard/form at 390 x 844.
Demo data size	Six fictional seeded records; not a large-dataset benchmark.
Response latency / throughput	Not measured; no p95 latency or requests-per-second result is claimed.
Concurrency / production load	Not tested. SQLite and Flask development-server behavior were not load-benchmarked.

The dashboard currently loads all of the account's applications to calculate counts and deadlines, then queries the filtered list. This is understandable for a small internship demo but can become inefficient for large datasets. Pagination, database aggregation, and indexed deadline queries should be evaluated before scaling. A production database and WSGI server would be needed for a hosted version.

The result supports a functioning local demonstration with verified core workflows. Further work should measure latency, memory use, concurrent access, and recovery behavior using a realistic dataset before making production-readiness claims.

7 7 My learnings

The completed project provides practical learning in connecting Python logic to a web interface and persistent data. The following learning summary is written for the student to review and personalize before submission.

Area	Learning demonstrated by the project
Python web development	Map requests to route functions and render HTML through Jinja templates.
Application organization	Use an application factory and blueprints instead of placing all features in one file.
Data modeling	Represent one-to-many ownership through User and Application tables.
Authentication / authorization	Understand the difference between being logged in and having permission to access a record.
Validation and safety	Validate dates/URLs, hash passwords, protect forms against CSRF, and escape user content.
Testing	Use isolated databases, parameterized invalid-input cases, and direct foreign-record requests.
Interface design	Provide useful empty states, explicit destructive-action confirmation, and responsive layouts.
Technical communication	Prepare reproducible setup instructions, evidence, diagrams, screenshots, and a clear demo.

One useful distinction is authentication versus authorization. A session identifies the logged-in user, but each application lookup still has to verify ownership. Another is input validation versus output escaping: accepting appropriate values and safely displaying text address different risks.

Testing the exact deadline boundaries also shows why business rules should be explicit. Today and the seventh day are included; past deadlines and the eighth day are excluded; Offered and Rejected records are not shown as upcoming active deadlines. This turns a vague feature description into reproducible behavior.

Personal reflection: [Add your own challenges, mentor feedback, training activities, and how this project supports your career goals. Include only experiences that actually occurred.]

8 8 Future work scope

Possible enhancement	Purpose / work required
Pagination and database aggregation	Improve dashboard behavior with many application records; benchmark before/after changes.
Email reminders	Add opt-in reminders, background scheduling, and notification delivery tests.
CSV import/export	Support data portability with validation and safe handling of spreadsheet content.
Status history	Record dated transitions and follow-up actions; currently only the latest status is stored.
Account management	Add email verification, password reset, and account settings.
Production deployment	Introduce a WSGI server, HTTPS, secure cookies, deployment secrets, database backups, and migrations.
Accessibility review	Extend automated/manual checks for contrast, screen readers, keyboard workflows, and error announcements.
Performance evaluation	Measure realistic dataset size, response latency, concurrent usage, and database contention.

These items are proposed extensions and are not included in the implemented version. The present result is a complete local application for internship evaluation, supported by tests, documentation, screenshots, and a silent walkthrough. Before submission, fill the cover details and repository links and review the personal learning narrative.